CodingLab5

May 24, 2026

Neural Data Science

Lecturer: Dr. Jan Lause, Prof. Dr. Philipp Berens

Tutors: Jonas Beck, Fabio Seel, Julius Würzler

Summer term 2025

Student names: Zhidong Zhang, Yuzhe Han, Bach Nguyen

LLM Disclaimer: Github Copilot was used to help with function calling and structure prompting, Claude and Gemini were used (by different members) to help with function understanding and outcome analysis.

1 Coding Lab 5

[ ]:
```
import matplotlib.pyplot as plt
import numpy as np
import scipy.optimize as opt
import scipy.io as io

%load_ext jupyter_black

%load_ext watermark
%watermark --time --date --timezone --updated --python --iversions --watermark␣
 ↪-p sklearn

%matplotlib inline
plt.style.use("../matplotlib_style.txt")
```

```
Last updated: 2026-05-24 21:55:54 CEST

Python implementation: CPython
Python version       : 3.10.0
IPython version      : 8.39.0

sklearn: 1.7.2

matplotlib: 3.10.8
numpy     : 2.2.6
```

```
scipy     : 1.15.3

Watermark: 2.6.0
```

2 Task 1: Fit RF on simulated data

We will start with toy data generated from an LNP model neuron to make sure everything works right. The model LNP neuron consists of one Gaussian linear filter, an exponential nonlinearity and a Poisson spike count generator. We look at it in discrete time with time bins of width δt. The model is:

$$c_t \sim Poisson(r_t) r_t = \exp(w^T s_t) \cdot \Delta t \cdot R$$

Here, c_t is the spike count in time window t of length Δt, s_t is the stimulus and w is the receptive field of the neuron. The receptive field variable `w` is 15×15 pixels and normalized to $||w|| = 1$. A stimulus frame is a 15×15 pixel image, for which we use uncorrelated checkerboard noise (binary) with a stimulus intesity of 5 (peak to peak). R can be used to bring the firing rate into the right regime (e.g. by setting $R = 50$).

For computational ease, we reformat the stimulus and the receptive field in a 225 by 1 array. The function `sample_lnp` can be used to generate data from this model. It returns a spike count vector `c` with samples from the model (dimensions: 1 by nT $= T/\Delta t$), a stimulus matrix `s` (dimensions: $225 \times$ nT) and the mean firing rate `r` (dimensions: nT $\times$ 1).

Here we assume that the receptive field influences the spike count instantaneously just as in the above equations. Implement a Maximum Likelihood approach to fit the receptive field.

To this end derive mathematically and implement the log-likelihood function $L(w)$ and its gradient $\frac{L(w)}{dw}$ with respect to w (`negloglike_lnp`). The log-likelihood of the model is

$$L(w) = \log \prod_t \frac{r_t^{c_t}}{c_t!} \exp(-r_t).$$

Make sure you include intermediate steps of the mathematical derivation in your answer, and you give as final form the maximally simplified expression, substituting the corresponding variables.

Plot the stimulus for one frame, the cell's response over time and the spike count vs firing rate. Plot the true and the estimated receptive field.

Grading: 2 pts (calculations) + 4 pts (generation) + 4 pts (implementation)

2.0.1 Calculations (2 pts)

You can add your calculations in LaTeX *here.*

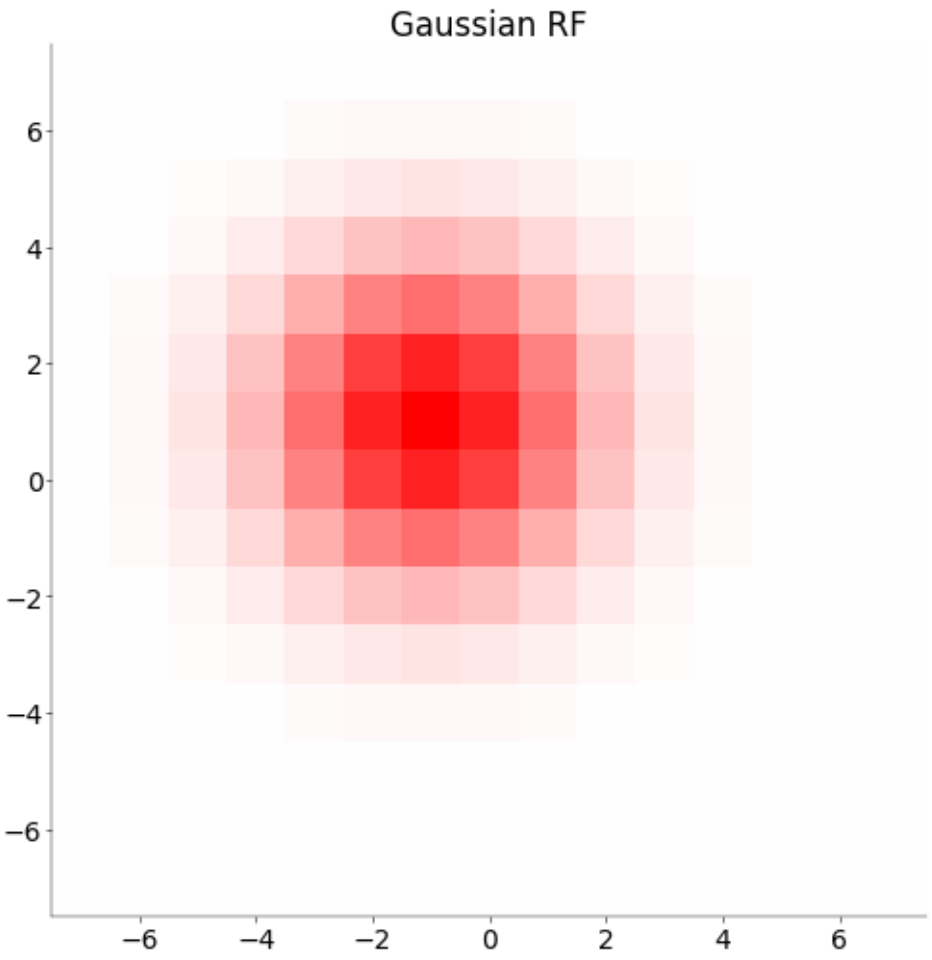
$$
\begin{aligned}
L(\omega) &= \log \prod_t \frac{r_t^{c_t}}{c_t!} \exp(-r_t) \\
&= \sum_t \log \left[\frac{r_t^{c_t}}{c_t!} \exp(-r_t) \right] \\
&= \sum_t \left[\log \left(\frac{r_t^{c_t}}{c_t!} \right) - r_t \right] \\
&= \sum_t \left[c_t \log(r_t) - \log(c_t!) - r_t \right] \\
&= \sum_t \left[c_t(\omega^\top s_t) + c_t \log(\Delta \cdot R) - \log(c_t!) - \exp(\omega^\top s_t) \cdot \Delta t \cdot R \right] \\
\frac{dL(\omega)}{d\omega} &= \sum_t \frac{d}{d\omega} \left[c_t(\omega^\top s_t) + c_t \log(\Delta \cdot R) - \log(c_t!) - \exp(w^\top s_t) \cdot \Delta t \cdot R \right] \\
&= \sum_t \frac{d}{d\omega} \left[c_t(\omega^\top s_t) - \exp(\omega^\top s_t) \cdot \Delta t \cdot R \right] \\
&= \sum_t \left[c_t s_t - \exp(\omega^\top s_t) \cdot \Delta t \cdot R \cdot s_t \right] \\
&= \sum_t \left[c_t - r_t \right] s_t
\end{aligned}
$$

2.0.2 Generate data (2 pts)

[ ]:
```python
def gen_gauss_rf(D: int, width: float, center: tuple = (0, 0)) -> np.ndarray:
    """
    Generate a Gaussian receptive field.

    Args:
        D (int): Size of the receptive field (DxD).
        width (float): Width parameter of the Gaussian.
        center (tuple, optional): Center coordinates of the receptive field. Defaults to (0, 0).

    Returns:
        np.ndarray: Gaussian receptive field.
    """

    sz = (D - 1) / 2
    x, y = np.meshgrid(np.arange(-sz, sz + 1), np.arange(-sz, sz + 1))
    x = x + center[0]
    y = y + center[1]
    w = np.exp(-(x**2 / width + y**2 / width))
    w = w / np.sum(w.flatten())

    return w
```

```
w = gen_gauss_rf(15, 7, (1, 1))
vlim = np.max(np.abs(w))
fig, ax = plt.subplots(1, 1, figsize=(5, 5))
ax.imshow(w, cmap="bwr", vmin=-vlim, vmax=vlim, extent=(-7.5, 7.5, -7.5, 7.5))
ax.set_title("Gaussian RF")
```

[ ]: Text(0.5, 1.0, 'Gaussian RF')



[ ]:
```
def sample_lnp(
    w: np.array, nT: int, dt: float, R: float, s_i: float, random_seed: int = 10
):
    """Generate samples from an instantaneous LNP model neuron with
    receptive field kernel w.

    Parameters
    ----------
```

```
    w: np.array, (Dx * Dy, )
        (flattened) receptive field kernel.

    nT: int
        number of time steps

    dt: float
        duration of a frame in s

    R: float
        rate parameter

    s_i: float
        stimulus intensity peak to peak

    random_seed: int
        seed for random number generator

    Returns
    -------

    c: np.array, (nT, )
        sampled spike counts in time bins

    r: np.array, (nT, )
        mean rate in time bins

    s: np.array, (Dx * Dy, nT)
        stimulus frames used

    Note
    ----

    See equations in task description above for a precise definition
    of the individual parameters.

    """

    rng = np.random.default_rng(random_seed)

    # ------------------------------------------------
    # Generate samples from an instantaneous LNP model
    # neuron with receptive field kernel w. (1 pt)
    # ------------------------------------------------
    c = []
    r = []
    s = rng.choice([-s_i / 2, s_i / 2], size=(w.shape[0], nT))
```

```
    # print("shape of s: ", s.shape)
    for i in range(nT):
        s_t = s[:, i]
        r_t = np.exp(w @ s_t) * dt * R
        c_t = rng.poisson(r_t)

        r.append(r_t)
        c.append(c_t)
    return np.array(c), np.array(r), s
```

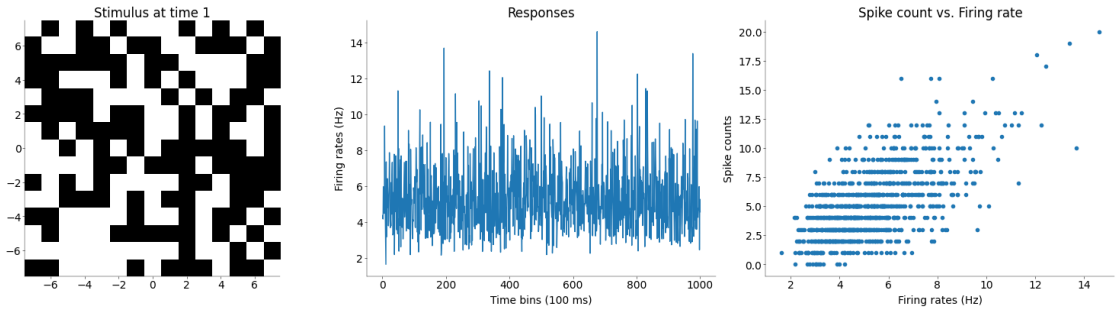
[ ]:
```
D = 15  # number of pixels
nT = 1000  # number of time bins
dt = 0.1  # bins of 100 ms
R = 50  # firing rate in Hz
s_i = 5  # stimulus intensity
extent = [-D / 2, D / 2, -D / 2, D / 2]

w = gen_gauss_rf(D, 7, (1, 1))
w = w.flatten()
c, r, s = sample_lnp(w, nT, dt, R, s_i)
```

Plot the stimulus for one frame, the cell's response over time and the spike count vs firing rate.

[ ]:
```
mosaic = mosaic = [["stim", "responses", "count/rate"]]

fig, axs = plt.subplot_mosaic(mosaic=mosaic, figsize=(15, 4))
#␣
 ↪-----------------------------------------------------------------------------------------
# Plot the stimulus for one frame, the cell's responses over time and spike␣
 ↪count vs firing rate (1 pt)
#␣
 ↪-----------------------------------------------------------------------------------------

ax = axs["stim"]
s_t1 = s[:, 0].reshape((D, D))
vlim = np.max(np.abs(s_t1))
ax.imshow(s_t1, cmap="gray", vmin=-vlim, vmax=vlim, extent=extent)
ax.set_title("Stimulus at time 1")

ax = axs["responses"]
ax.plot(np.arange(1, nT + 1), r)
ax.set_title("Responses")
ax.set_ylabel("Firing rates (Hz)")
ax.set_xlabel("Time bins (100 ms)")

ax = axs["count/rate"]
ax.scatter(r, c)
```

```
ax.set_title("Spike count vs. Firing rate")
ax.set_ylabel("Spike counts")
ax.set_xlabel("Firing rates (Hz)")
```

[ ]:
```
Text(0.5, 0, 'Firing rates (Hz)')
```



2.0.3 Implementation (3 pts)

Implement the negative log-likelihood of the LNP and its gradient with respect to the receptive field using the simplified equations you calculated earlier *(1 pt)*

[ ]:
```
def negloglike_lnp(
    w: np.array, c: np.array, s: np.array, dt: float = 0.1, R: float = 50
) -> float:
    """Implements the negative (!) log-likelihood of the LNP model

    Parameters
    ----------

    w: np.array, (Dx * Dy, )
      current receptive field

    c: np.array, (nT, )
      spike counts

    s: np.array, (Dx * Dy, nT)
      stimulus matrix


    Returns
    -------

    f: float
      function value of the negative log likelihood at w
```

```
    """

    # ------------------------------------------------
    # Implement the negative log-likelihood of the LNP
    # ------------------------------------------------

    from scipy.special import factorial

    r_t = np.exp(w @ s) * dt * R
    f = -np.sum(c * np.log(r_t) - np.log(factorial(c)) - r_t)

    return f


def deriv_negloglike_lnp(
    w: np.array, c: np.array, s: np.array, dt: float = 0.1, R: float = 50
) -> np.array:
    """Implements the gradient of the negative log-likelihood of the LNP model

    Parameters
    ----------

    see negloglike_lnp

    Returns
    -------

    df: np.array, (Dx * Dy, )
      gradient of the negative log likelihood with respect to w

    """

    # ---------------------------------------------------------------
    # Implement the gradient with respect to the receptive field `w`
    # ---------------------------------------------------------------

    r_t = np.exp(w @ s) * dt * R
    df = -s @ (c - r_t)

    return df
```

The helper function `check_grad` in `scipy.optimize` can help you to make sure your equations and implementations are correct. It might be helpful to validate the gradient before you run your optimizer.

[ ]:
```
# Check gradient
opt.check_grad(negloglike_lnp, deriv_negloglike_lnp, w, c, s)
```

[ ]:
```
np.float64(0.0038122178434290012)
```

Fit receptive field maximizing the log likelihood.

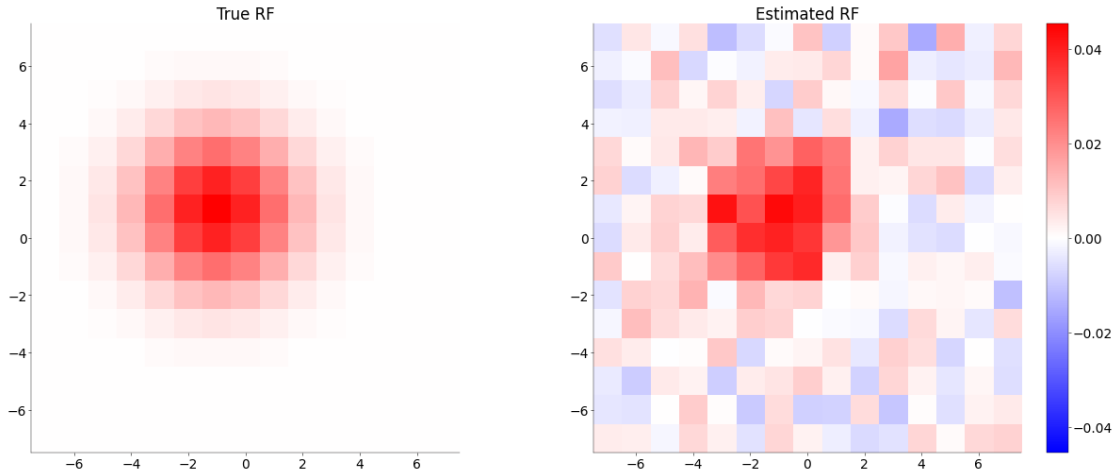
The scipy.optimize package also has suitable functions for optimization. If you generate a large number of samples, the fitted receptive field will look more similar to the true receptive field. With more samples, the optimization takes longer, however.

[ ]:
```
# ----------------------------------------
# Estimate the receptive field by maximizing
# the log-likelihood (or more commonly,
# minimizing the negative log-likelihood).
#
# Tips: use scipy.optimize.minimize(). (1 pt)
# ----------------------------------------

res = opt.minimize(
    fun=negloglike_lnp,
    x0=np.ones_like(w) * 1e-3,
    args=(c, s, dt, R),
    jac=deriv_negloglike_lnp,
)
```

Plot the true and the estimated receptive field.

[ ]:
```
# ----------------------------------
# Plot the ground truth and estimated
# `w` side by side. (1 pt)
# ----------------------------------

mosaic = [["True", "Estimated"]]
fig, ax = plt.subplot_mosaic(mosaic=mosaic, figsize=(12, 5))

# make sure to add a colorbar. 'bwr' is a reasonable choice for the cmap.
w_fit = res.x

vlim = np.max(np.abs(w))

ax["True"].imshow(w.reshape((D, D)), cmap="bwr", vmin=-vlim, vmax=vlim,␣
 ↪extent=extent)
ax["True"].set_title("True RF")
im_est = ax["Estimated"].imshow(
    w_fit.reshape((D, D)), vmin=-vlim, vmax=vlim, cmap="bwr", extent=extent
)
fig.colorbar(im_est, ax=ax["Estimated"])
ax["Estimated"].set_title("Estimated RF")
```

[ ]:
```
Text(0.5, 1.0, 'Estimated RF')
```



3 Task 2: Apply to real neuron

Download the dataset for this task from Ilias (`nds_cl_5_data.mat`). It contains a stimulus matrix (`s`) in the same format you used before and the spike times. In addition, there is an array called `trigger` which contains the times at which the stimulus frames were swapped.

- Generate an array of spike counts at the same temporal resolution as the stimulus frames
- Fit the receptive field with time lags of 0 to 4 frames. Fit them one lag at a time (the ML fit is very sensitive to the number of parameters estimated and will not produce good results if you fit the full space-time receptive field for more than two time lags at once).
- Plot the resulting filters

Grading: 3.5 pts

[ ]:
```
var = io.loadmat("../data/nds_cl_5_data.mat")

# t contains the spike times of the neuron
t = var["DN_spiketimes"].flatten()

# trigger contains the times at which the stimulus flipped
trigger = var["DN_triggertimes"].flatten()

# contains the stimulus movie with black and white pixels
s = var["DN_stim"]
s = s.reshape((300, 1500))  # the shape of each frame is (20, 15)
s = s[:, 1 : len(trigger)]
```

Create vector of spike counts

[ ]:
```
# ----------------------------------------
# Bin the spike counts at the same temporal
# resolution as the stimulus (0.5 pts)
```

```
# ----------------------------------------
c, _ = np.histogram(t, bins=trigger)
print("Spike count shape:", c.shape)
print("Stimulus shape:", s.shape)
```

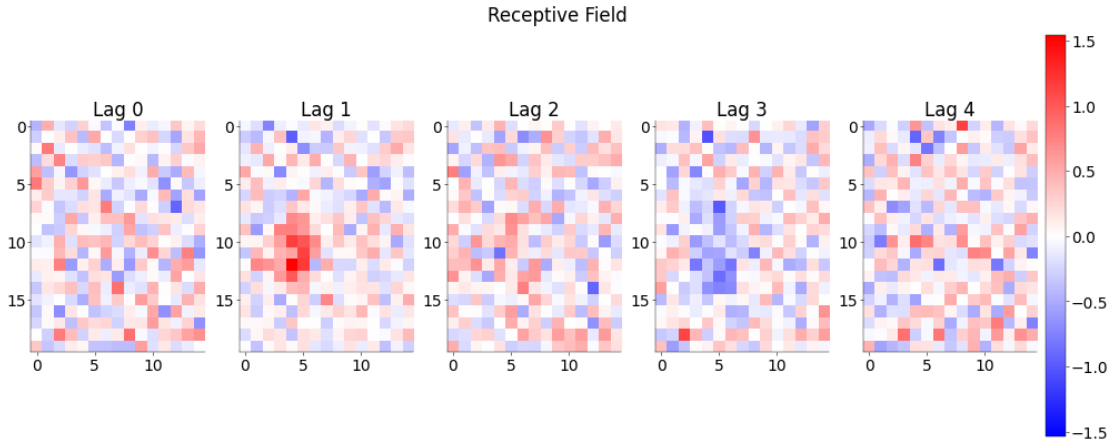
```
Spike count shape: (1488,)
Stimulus shape: (300, 1488)
```

Fit receptive field for each frame separately

[ ]:
```
# ----------------------------------------
# Fit the receptive field with time lags of
# 0 to 4 frames separately (1 pt)
#
# The final receptive field (`w_hat`) should
# be in the shape of (Dx * Dy, 5)
# ----------------------------------------

# specify the time lags
delta = [0, 1, 2, 3, 4]
Dx_Dy = s.shape[0]

w_hat = np.zeros((Dx_Dy, len(delta)))

s_centered = s - np.mean(s)

# fit for each delay
for i, delt in enumerate(delta):
    if delt != 0:
        c_lag = c[delt:]
        s_lag = s_centered[:, :-delt]
    else:
        c_lag = c
        s_lag = s_centered

    # fitting
    res = opt.minimize(
        fun=negloglike_lnp,
        x0=np.ones(shape=(Dx_Dy,)) * 1e-3,
        args=(c_lag, s_lag, dt, 1),  # R=1 (larger R would induce large␣
 ↪intercept of RF)
        jac=deriv_negloglike_lnp,
        method="BFGS",
        options={"maxiter": 300},
    )
    w_hat[:, i] = res.x
```

Plot the frames one by one and explain what you see.

```
[ ]: # ------------------------------------------
     # Plot all 5 frames of the fitted RFs (1 pt)
     # ------------------------------------------
     fig, ax = plt.subplot_mosaic(mosaic=[delta], figsize=(10, 4),␣
      ↪constrained_layout=True)

     fig.suptitle("Receptive Field")
     vlim = max([np.max(np.abs(w)) for w in w_hat])
     for i in delta:
         im = ax[i].imshow(w_hat[:, i].reshape((20, 15)), cmap="bwr", vmin=-vlim,␣
      ↪vmax=vlim)
         ax[i].set_title(f"Lag {i}")
         if i == 4:
             fig.colorbar(im, ax=ax[i])
```



Explanation (1 pt)

The receptive field for 1-time lags shows a positive centers prererence, while the receptive field for 3-time lags shows a negative centers preference at the same location. It suggests that the neuron is not equally sensitive to all stimulus delays. Instead, it appears to be driven more strongly by stimulus features occurring at specific lags before the spike, while the exact sign of bright versus dark pixels is encoded by positive and negative RF weights. Moreover, the fitted filters are noisy because each lag is estimated from a limited number of stimulus-response pairs and 300 spatial parameters. The separate fits are easier to estimate than one full 300 x 5 parameter spatio-temporal model, but they also ignore correlations between adjacent lags.

4 Task 3: Separate space/time components

The receptive field of the neuron can be decomposed into a spatial and a temporal component. Because of the way we computed them, both are independent and the resulting spatio-temporal component is thus called separable. As discussed in the lecture, you can use singular-value decomposition to separate these two:

$$W = u_1 s_1 v_1^T$$

Here u_1 and v_1 are the singular vectors belonging to the 1st singular value s_1 and provide a long rank approximation of W, the array with all receptive fields. It is important that the mean is subtracted before computing the SVD.

Plot the first temporal component and the first spatial component. You can use a Python implementation of SVD. The results can look a bit puzzling, because the sign of the components is arbitrary.

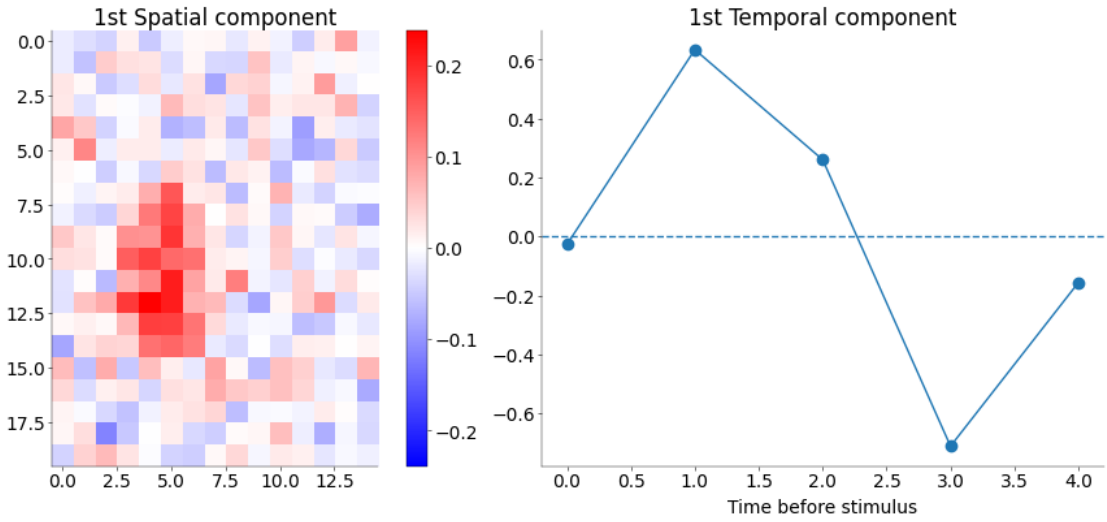
Grading: 1.5 pts

[ ]:
```
# ------------------------------------------
# Apply SVD to the fitted receptive field,
# you can use either numpy or sklearn (0.5 pt)
# ------------------------------------------
w_hat = w_hat.T
w_hat_centerd = w_hat - w_hat.mean(axis=0)
U, S, Vh = np.linalg.svd((w_hat_centerd))
u1 = U[:, 0]
s1 = S[0]
v1 = Vh[0, :].transpose()

np.set_printoptions(suppress=True, precision=6)
print("Sigmas: ", S)
```

```
Sigmas:  [5.584855 4.62026  4.424877 3.928092 0.      ]
```

[ ]:
```
# ------------------------------------------------
# Plot the spatial and temporal components (1 pt)
# ------------------------------------------------

fig, ax = plt.subplot_mosaic(
    mosaic=[["Spatial", "Temporal"]], figsize=(10, 4), constrained_layout=True
)
# add plot

vlim = np.max(np.abs(v1))
im = ax["Spatial"].imshow(v1.reshape((20, 15)), cmap="bwr", vmin=-vlim,␣
 ↪vmax=vlim)
ax["Spatial"].set_title("1st Spatial component")
fig.colorbar(im, ax=ax["Spatial"])
ax["Temporal"].plot(u1, marker="o")
ax["Temporal"].axhline(0, linestyle="--")
ax["Temporal"].set_title("1st Temporal component")
ax["Temporal"].set_xlabel("Time before stimulus")
# fig.colorbar(im, ax=ax["Spatial"])
```

[ ]: `Text(0.5, 0, 'Time before stimulus')`



5 Task 4: Regularized receptive field

As you can see, maximum likelihood estimation of linear receptive fields can be quite noisy, if little data is available.

To improve on this, one can regularize the receptive field vector and a term to the cost function

$$C(w) = L(w) + \alpha||w||_p^2$$

Here, the p indicates which norm of w is used: for $p = 2$, this is shrinks all coefficient equally to zero; for $p = 1$, it favors sparse solutions, a penality also known as lasso. Because the 1-norm is not smooth at zero, it is not as straightforward to implement "by hand".

Use a toolbox with an implementation of the lasso-penalization and fit the receptive field. Possibly, you will have to try different values of the regularization parameter α. Plot your estimates from above and the lasso-estimates. How do they differ? What happens when you increase or decrease *alpha*?

If you want to keep the Poisson noise model, you can use the implementation in `pyglmnet`. Otherwise, you can also resort to the linear model from `sklearn` which assumes Gaussian noise (which in my hands was much faster).

Grading: 3 pts

[ ]:
```
from sklearn import linear_model
from pyglmnet import GLM

# ----------------------------------------
```

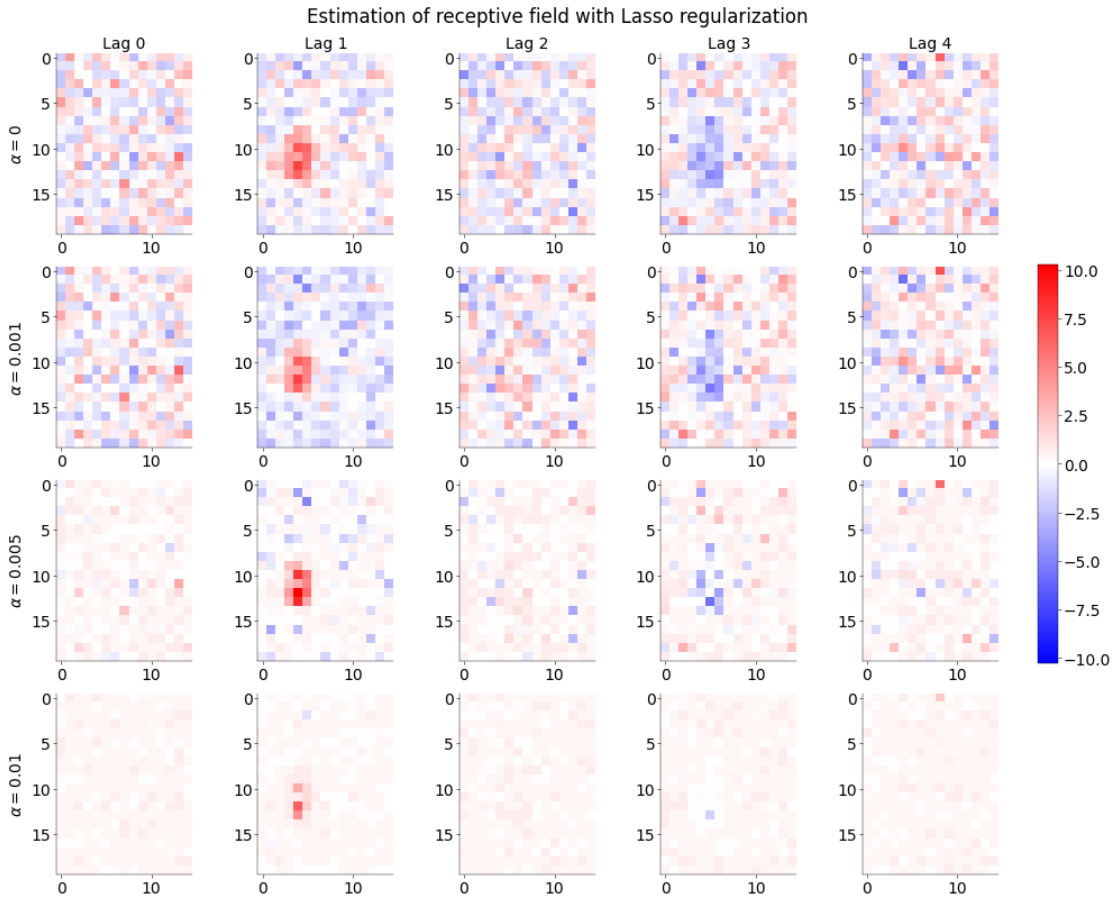
```
# Fit the receptive field with time lags of
# 0 to 4 frames separately (the same as before)
# with sklearn or pyglmnet for different values
# of alpha (1 pt)
# ----------------------------------------

delta = [0, 1, 2, 3, 4]
alphas = [0, 1e-3, 5 * 1e-3, 1e-2]

w_fit_all = []

for alpha in alphas:
    ws = []
    for delt in delta:
        if delt != 0:
            c_lag = c[delt:]
            s_lag = s[:, :-delt]
        else:
            c_lag = c
            s_lag = s
        clf = GLM("poisson", alpha=1, reg_lambda=alpha)
        clf.fit(s_lag.T, c_lag)
        ws.append(clf.beta_)
        del clf
    w_fit_all.append(ws)
```

```
/Users/eastos/miniforge3/envs/nds_env/lib/python3.10/site-
packages/pyglmnet/pyglmnet.py:863: UserWarning: Reached max number of iterations
without convergence.
  warnings.warn(
```

[ ]:
```
# ----------------------------------------
# plot the estimated receptive fields (1 pt)
# ----------------------------------------
w_fit_all = np.array(w_fit_all)
fig, ax = plt.subplots(
    len(alphas), len(delta), figsize=(10, 8), constrained_layout=True
)  # add plot

vlim = max([max([np.max(w) for w in ws]) for ws in w_fit_all])
for i in range(len(alphas)):
    for j in range(len(delta)):
        im = ax[i, j].imshow(
            w_fit_all[i, j].reshape((20, 15)), cmap="bwr", vmin=-vlim, vmax=vlim
        )
        if i == 0:
            ax[i, j].set_title(f"Lag {delta[j]}", fontsize=10)
```

```
        if j == 0:
            ax[i, j].set_ylabel(f"$\\alpha={alphas[i]}$")

cbar = fig.colorbar(im, ax=ax, orientation="vertical", fraction=0.02, pad=0.04)

fig.suptitle("Estimation of receptive field with Lasso regularization")
```

[ ]: Text(0.5, 0.98, 'Estimation of receptive field with Lasso regularization')



Explanation (1 pt)

Compared with the unregularized maximum-likelihood estimates, the lasso-regularized receptive fields show a clearer sparsity pattern due to the L1 penalty shrinking down the irrelevant values to near zero. With α increasing, the estimated receptive field becomes sparser with more values being near zero, which make it less noisy and remain the major features shown in time-1 lag (positive preference) and time-3 lag (negative preference).

5.1 Bonus Task (Optional): Spike Triggered Average

Instead of the Maximum Likelihood implementation above, estimate the receptive field using the spike triggered average. Use it to increase the temporal resolution of your receptive field estimate. Perform the SVD analysis for your STA-based receptive field and plot the spatial and temporal kernel as in Task 3.

Questions: 1. Explain how / why you chose a specific time delta. 2. Reconsider what you know about STA. Is it suitable to use STA for this data? Why/why not? What are the (dis-)advantages of using the MLE based method from above?

Grading: 1 BONUS Point.

BONUS Points do not count for this individual coding lab, but sum up to 5% of your ***overall coding lab grade****. There are 4 BONUS points across all coding labs.*

[ ]: